I chose Microservices Architecture for the Database Query Performance Profiler System.

Architectural Choice

Microservices Architecture. I split the system into separate services for log ingestion, plan parsing, index recommendation, digest reporting, and recommendation access, each communicating through defined interfaces.

Reason 1

The system needs to handle up to 5,000 log records per minute (NFR-001). With microservices, the Log Ingestion Service can scale on its own during peak load without having to scale the other services too, since they don't get nearly as much traffic.

Reason 2

The index recommendation logic is more complex and more likely to break than the rest of the system. If it fails, ingestion can keep running and logs don't get lost, and the digest can still go out based on whatever recommendations were already generated.

Security Advantage

Each service only has access to what it needs. The Log Ingestion Service can write logs but can't touch the Metadata & Recommendation Store directly, and only the Recommendation Access API is exposed to the Database Administrator and Backend Lead, matching NFR-002. So if one service gets compromised, it doesn't automatically expose everything else.

Performance Benefit

Since ingestion doesn't wait on analysis, new logs can keep coming in even while the Index Recommendation Engine is still working through an earlier batch. That separation is what makes hitting the 5,000 records/minute requirement realistic.